GUIDE

Developer Guide

Working on the shared development box

PREPARED FOR

Your organisation

Everyone with an account on a development box

Who this is for	**Staff and developers — no Linux experience assumed**
Written for	**Windows with Git Bash; macOS differences noted**
What you need	**A laptop, a VPN config, an account**
Access	**SSH over the WireGuard VPN, or the campus LAN**
Companion documents	**Hardware Specification · Setup Guide**

PT ArtiVisi Intermedia
Software engineering & technical training
info@artivisi.com · artivisi.com

19 August 2026

How to read this guide

The guide is in four parts and they get progressively more technical. Most people never need the last one.

Part	What it covers	Read it if
1	Why this box exists and how the pieces fit together	Always — it is four pictures and takes ten minutes
2	Recipes: step-by-step for everything you do regularly	Always — this is the part you will come back to
3	Reference: the full command lists, what is on the box, the limits	When you need a command you have forgotten
4	Building and running applications on the box	Only if you write software. Staff editing documents and websites can stop after Part 3

Commands you type are shown like this, with the machine you type them on shown before the `$`:

```
laptop$ ssh box-01          # typed on your own laptop
box-01$ ls                  # typed on the box, after you are connected
```

You never type the `laptop$` or `box-01$` part — it is only there to tell you **where** you are. Getting those two confused is the single most common early mistake, and Part 1 explains why they are different machines at all.

This guide is written for Windows, because that is what most people here use. Windows does not come with the tools, so Recipe 1 installs them — that is what Git Bash is for, and every command in this guide is written to run in it. Where macOS differs, it is noted. Once you are connected to the box, the two are identical: the box runs Linux either way, and from that point the guide makes no distinction.

Part 1 — Why this box exists

Where you are now

Today you work like this: Claude Code runs on your own laptop, in a terminal, on one folder that holds one project.

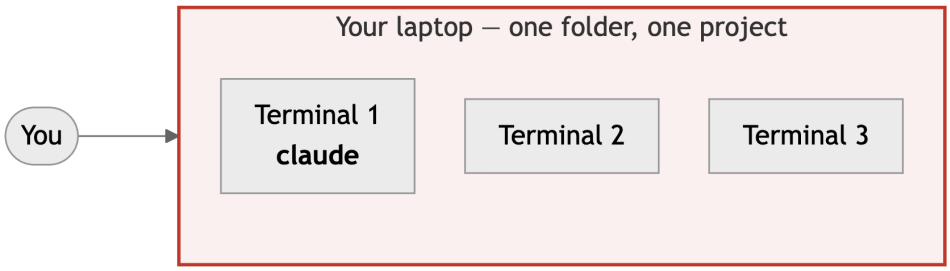

Figure 1: Today — everything runs on your own laptop. One folder, one project. Close the lid and it all stops.

This works, and for small edits it is still the right thing. It has three limits that get worse as the work grows:

- **Close the laptop and everything stops.** A long job — Claude Code working through a large document set, a website build — dies with it.
- **Your laptop is the only place the work exists.** Nobody else can see it or pick it up.
- **One folder, one project.** Working on the website and a curriculum document at the same time means two windows, two folders, and remembering which is which.

Moving the work to a box that never sleeps

The development box is an ordinary PC that stays on all the time. You keep using your own laptop; the difference is **where the work actually runs**.

Figure 2: `ssh` is how your laptop reaches the box. The terminals — and the work in them — are on the box, not on your laptop.

`ssh` means "secure shell". It gives you a terminal on another computer. What you type goes to the box; what the box prints comes back to your screen. Your laptop becomes a window onto the box rather than the place the work happens.

That solves the second and third problems immediately: the work is on a machine everyone can reach, and the box is big enough to hold several projects at once. But on its own it does **not** solve the first one.

The problem `ssh` does not solve

Close the laptop and the work still dies. When your `ssh` connection drops — you close the lid, the Wi-Fi hiccups, you go home — the box notices that the terminal on the other end has gone. It shuts that terminal down, and everything running inside it stops with it. A Claude Code session that was halfway through a job is simply gone.

This surprises people, because it feels like the work is "on the server" and should be safe. It is on the server, but it belongs to a terminal, and the terminal belonged to your connection.

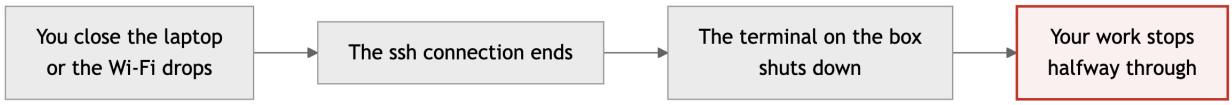

Figure 3: Without tmux: disconnecting kills the terminal, and killing the terminal kills the work.

tmux — the part that makes it work

`tmux` is a program that runs **on the box** and holds your terminals for you. You connect to tmux instead of connecting straight to a bare terminal.

The difference is who owns the terminals. Without tmux they belong to your connection, so they die with it. With tmux they belong to tmux, which is running on the box and does not care whether you are connected.

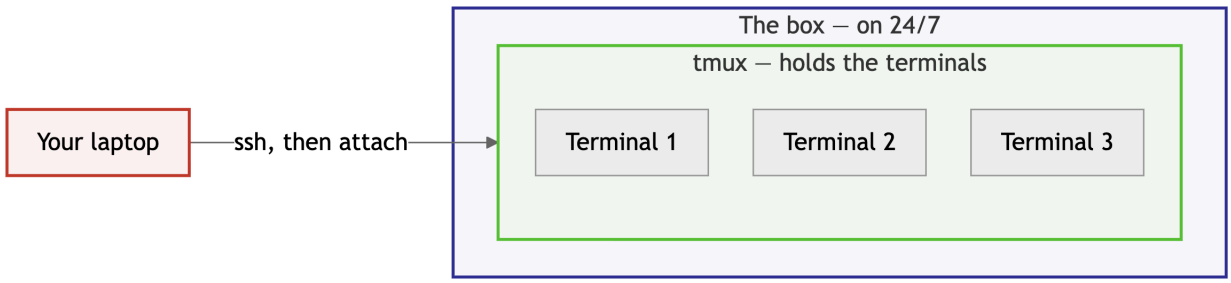

Figure 4: With tmux: you attach and detach freely. The terminals stay running on the box whether or not your laptop is connected.

So the working day becomes: connect, **attach** to tmux, do some work, **detach**, close the laptop and go home. Tomorrow you connect, attach again, and everything is exactly as you left it — the same terminals, the same Claude Code sessions, output and all.

This is the single idea worth taking away from Part 1. Everything else is detail. If you remember only one thing: **always work inside tmux**. A terminal outside tmux is one dropped connection away from losing your work.

The whole picture

Now put several people on the same box, and add the place the work is **shared**.

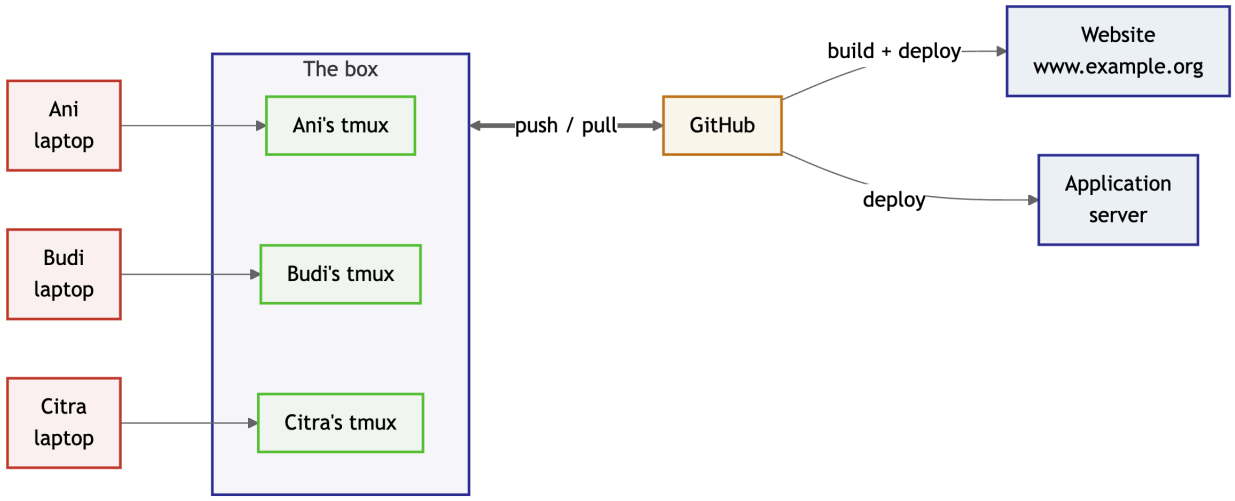

Figure 5: Everyone has their own account and their own tmux on the same box. Work is shared through GitHub, not by looking at each other's files.

Three things are worth noticing:

1. **Each person has their own separate space on the box.** You cannot see other people's files and they cannot see yours, even though you share the machine. Your tmux, your folders, your Claude Code sessions are yours alone.
2. **You share work through GitHub, not through the box.** You `push` your finished changes up; other people `pull` them down. This is why two people can edit the same document set without overwriting each other.
3. **Publishing is automatic.** When approved work reaches GitHub, the website rebuilds and deploys itself. Nobody copies files to a server by hand.

So the full loop for a piece of work is: connect to the box, attach to tmux, `pull` the latest, make your change, `push` it, and the rest happens without you. Part 2 is that loop, one recipe at a time.

Part 2 — Recipes

Each recipe is a complete task, start to finish. Follow the steps in order. The first two you do once; the rest you will do every working day.

Recipe 1 — Set up your own laptop (once)

Everything here happens on **your own laptop**, before the box is involved at all. It takes about half an hour and you never do it again. Do not skip steps: each one is needed by a later one.

Step 1 — Install Git

Git is the program that records changes and moves them between your machine, the box, and GitHub. On Windows it also installs **Git Bash**, which is the terminal you will use for everything else.

Windows. Download from git-scm.com/download/win and run the installer. Accept every default — there are many screens and none of them need changing. When it finishes you will have a new program in the Start menu called **Git Bash**.

macOS. Open the Terminal app and type `git --version`. If it offers to install developer tools, accept. Nothing else to do.

Step 2 — Open your terminal

Windows: Start menu → **Git Bash**.
macOS: Applications → Utilities → **Terminal**.

You get a window with a line of text ending in `$`. That is the **prompt**: it means the computer is waiting for you to type a command. You type a command, press Enter, and it does it.

Check Git arrived:

```
laptop$ git --version
git version 2.47.1
```

If you see a version number, Step 1 worked.

> **Copy and paste work differently here.** In Git Bash, `Ctrl-C` does **not** copy — it interrupts whatever is running, which is occasionally what you want and usually not. Use:
>
> - **Copy:** select text with the mouse — selecting is enough, or right-click → Copy.
> - **Paste:** right-click, or `Shift-Insert`.
>
> This catches everyone once. When you paste the long key line in Step 5, use right-click.

> **Use Git Bash, not PowerShell or Command Prompt,** if you are on Windows. The commands in this guide are written for it, and the other two use different syntax for paths and quoting. Everything here assumes the `$` prompt of Git Bash.

Step 3 — Tell Git who you are

Every change you record carries your name and email, so colleagues can see who did what. Set it once:

```
laptop$ git config --global user.name "Your Full Name"
laptop$ git config --global user.email "you@example.org"
```

Use your real name and your work email. Check it took:

```
laptop$ git config --global --list
user.name=Your Full Name
user.email=you@example.org
```

Step 4 — Create your key pair

A **key pair** replaces a password. It is two files that belong together: one you keep and one you hand out. Nothing you type is ever sent anywhere, which is why it is safer than a password.

```
laptop$ ssh-keygen -t ed25519 -C "you@example.org"
```

It asks three things:

1. **"Enter file in which to save the key"** — press Enter to accept the default. Do not type a name here; later steps expect the default location.
2. **"Enter passphrase"** — press Enter, leaving it empty.
3. **"Enter same passphrase again"** — press Enter again.

Three presses of Enter in total, then. Leaving the passphrase empty is a deliberate decision here, not a shortcut: with one you would type it several times a day, and forgetting it produces a support ticket that ends in the key being revoked and replaced — the same work as a lost laptop, arrived at by a more annoying route. A stolen laptop is handled by revoking the key instead, which is quick and has to exist as a capability either way.

What that trades away is real and worth knowing: anyone who obtains the file can use it until it is revoked. Which is why the next paragraph matters.

> **The trade-off, stated plainly.** Anyone who obtains that file can act as you until the key is revoked. If your laptop is ever lost or stolen, that is what Recipe 9 is for — read it now so you know it exists, and act the same hour if it ever happens.

You will see something like:

```
Your identification has been saved in /c/Users/YourName/.ssh/id_ed25519
Your public key has been saved in /c/Users/YourName/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:IuwIGkbxeKJmR5+xSLoElyZGDaGkQCaJLEjRHXLjsbs you@example.org
```

Those two file names are the whole point of the next step.

Step 5 — Know which key is which

This is the single most important thing on this page. You now have two files, in a hidden folder called `.ssh` inside your home directory:

File	Which	What you do with it
`id_ed25519`	**Private**	**Never** send this to anyone. Not to me, not to GitHub, not in WhatsApp, not in email. It never leaves your laptop. Anyone who has it can act as you.
`id_ed25519.pub`	**Public**	Safe to share with anyone. This is the one you register with GitHub and send to me. `.pub` is short for public.

Where they are:

System	Folder
Windows	`C:\Users\YourName\.ssh\` — in Git Bash, `~/.ssh/`
macOS	`/Users/yourname/.ssh/` — also `~/.ssh/`

`~` is shorthand for your home folder, and a folder starting with `.` is hidden — Windows Explorer and Finder will not show it unless you ask them to. That is normal. Use the terminal to look:

```
laptop$ ls ~/.ssh/
id_ed25519      id_ed25519.pub      known_hosts
```

Now print the **public** one. Note the `.pub` at the end — this is the only key you ever copy:

```
laptop$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICIYjZ2pxHTXpFjr/cMf9mx0+7UZdNElilCQvZvdDa2w
you@example.org
```

Select that whole line and copy it. It is one line, beginning `ssh-ed25519` and ending with your email. You need it twice, in the next two steps.

> **How to tell instantly whether you are about to leak the private key.** The public key is **one short line** starting `ssh-ed25519`. The private key is **many lines** between `-----BEGIN OPENSSH PRIVATE KEY-----` and `-----END OPENSSH PRIVATE KEY-----`. If what you are about to paste has BEGIN and END lines, stop — that is the wrong file.

Step 6 — Register the public key with GitHub

This is what lets you download projects and send your work back.

1. Go to github.com/settings/keys (log in first).
2. Click **New SSH key**.
3. **Title:** something that identifies the machine, like `Laptop kantor`.
4. **Key type:** Authentication Key.
5. **Key:** paste the line you copied in Step 5.
6. Click **Add SSH key**.

Check it worked:

```
laptop$ ssh -T git@github.com
```

The first time it asks whether to trust GitHub — type `yes`. Then you should see:

```
Hi yourname! You've successfully authenticated, but GitHub does not provide shell
access.
```

That sentence looks like an error. It is not — "does not provide shell access" is normal and expected. Seeing your own username is the success signal.

Step 7 — Send the same public key for box access

GitHub and the development box are two separate systems and neither knows about the other, so the same public key must be registered in both places.

Send **the same single line** from Step 5 to whoever runs the box, by email or chat. Say which machine it is from, in case you later add a second laptop.

You are sending the contents of `id_ed25519.pub`. If what you paste has BEGIN and END lines in it, you have opened the wrong file — see Step 5.

Step 8 — Install the VPN

The box is not reachable from the open internet. The VPN puts your laptop on the same private network.

1. Install WireGuard from wireguard.com/install — **Windows** or **macOS** as appropriate.
2. Ask for your `.conf` file. It is personal to your laptop; do not share it or reuse someone else's.
3. Open WireGuard → **Import tunnel(s) from file** → choose the file.
4. Click **Activate**.

On campus you can reach the box without this. Connect it anyway, so it is already working the first time you try from home.

Step 9 — Check the whole chain

You are waiting on one thing from another person — your account on the box — so confirm what you can:

Check	Expected
`git --version`	a version number
`git config --global user.name`	your name
`ls ~/.ssh/`	`id_ed25519` and `id_ed25519.pub`
`ssh -T git@github.com`	`Hi yourname! You've successfully authenticated`
WireGuard shows **Active**	a green or connected state

When you are told your account is ready, go to Recipe 2.

Recipe 2 — Connect to the box for the first time

1. **Activate the VPN** (unless you are on campus). WireGuard → your tunnel → **Activate**.

2. **Connect.** Use the username and address you were given:

```
laptop$ ssh yourname@10.9.0.10        # your box's address, from hosts.ini
```

   The first time, it asks whether you trust the machine. Type `yes`:

```
The authenticity of host '10.9.0.10' can't be established.
ED25519 key fingerprint is SHA256:F4GMNJjj9PXKbGdQP8HNS9jDN35ULmNoARTp8pARqmQ.
Are you sure you want to continue connecting (yes/no)? yes
```

   It will not ask for a password — your key is what identifies you. You should land on a prompt showing the box's name:

```
yourname@box-01:~$
```

   You are now typing **on the box**. Every command goes there, not to your laptop.

3. **Make it shorter.** On your laptop, create or edit `~/.ssh/config`:

```
Host box-01
    HostName 10.9.0.10
    User yourname
```

   Now `ssh box-01` is enough.

4. **Tell Git who you are — again, on the box.** Step 3 configured your laptop; the box is a different computer:

```
box-01$ git config --global user.name "Your Full Name"
box-01$ git config --global user.email "you@example.org"
```

5. **Log in to Claude Code, once:**

```
box-01$ claude
```

   Use **your own** Claude account. The box holds no shared login.

> **If `ssh` hangs or says "connection refused",** the VPN is almost certainly not connected. Check WireGuard before anything else — it is the cause nine times out of ten.

Recipe 3 — Start working on a project

Do this once per project. It copies the project from GitHub onto the box.

1. **Connect and start tmux.** From now on, every session starts this way:

```
laptop$ ssh box-01
box-01$ tmux new -As handbook
```

   `tmux new -As <name>` means "attach to the tmux session called `<name>`, or create it if it does not exist". Use one session per project; the name is yours to choose.

2. **Get the project.** Repositories live in `~/src`:

```
box-01$ cd ~/src
box-01$ git clone git@github.com:your-org/handbook.git
box-01$ cd handbook
```

`git clone` downloads the whole project and its history. You only do this once per project per box.

3. **Start work.**

```
box-01$ claude
```

Now you have a project on the box, inside tmux, with Claude Code running.

Recipe 4 — Make a change and share it

This is the recipe you will use most. Four commands, always in the same order.

1. **Start from everyone else's latest work.** Always, before you change anything:

```
box-01$ cd ~/src/handbook
box-01$ git pull
```

2. **Make your change** — edit files yourself, or ask Claude Code to.
3. **See what you changed.** This is a safety check, not a formality:

```
box-01$ git status
```

```
Changes not staged for commit:
        modified:   kurikulum/rps-basis-data.md
```

If a file you did not expect appears here, stop and find out why before continuing.

4. **Record the change with a message describing it:**

```
box-01$ git add .
box-01$ git commit -m "Update RPS Basis Data for semester ganjil 2026"
```

`git add .` marks everything you changed; `git commit` records it with your message. The message is read by colleagues later — write what changed and why, not "update".

5. **Send it to GitHub, where everyone else can get it:**

```
box-01$ git push
```

Until you `push`, your change exists only on the box and nobody else can see it. After `push`, it is on GitHub, and — for the website — the rebuild and deploy happen on their own.

> **`commit` and `push` are two different things,** and this trips up almost everyone at first. `commit` saves a checkpoint **on the box**. `push` sends those checkpoints **to GitHub**. You can commit ten times and push once. Work that is committed but not pushed is invisible to your colleagues.

Recipe 5 — Get your colleagues' changes

```
box-01$ cd ~/src/handbook
box-01$ git pull
```

Do this at the start of every working session, and again before you `push`. Pulling often is how you avoid conflicts; pulling rarely is how you create them.

If `git pull` reports a **conflict**, it means you and someone else changed the same lines of the same file, and git cannot decide which to keep. Do not guess and do not delete anything. Ask Claude Code:

> `git pull` reported a conflict in `kurikulum/rps-basis-data.md`. Show me both versions, explain what each side changed, and help me combine them.

Recipe 6 — Leave work running and come back to it

This is the reason the box exists. You can start something long, disconnect, go home, and pick it up exactly where it was.

To leave, with everything still running:

1. Press `Ctrl-b`, release both keys, then press `d`.

   You are back at your laptop's own prompt. Everything on the box keeps running — Claude Code included.

2. Close the laptop. Go home.

To come back:

```
laptop$ ssh box-01
box-01$ tmux attach
```

Your screen is exactly as you left it, including anything that finished while you were away.

> **`Ctrl-b` is tmux's "get ready" key.** It is not a command by itself — it tells tmux the **next** key is for it and not for the program you are using. `Ctrl-b` then `d` means detach. Press them one after the other, not together. Everything tmux does works this way, which is why the reference in Part 3 is a list of `Ctrl-b` plus one letter.

Recipe 7 — Work on two things at once

Each piece of work gets its own **window** inside your tmux session. Windows are like tabs: you switch between them instantly, and the ones you are not looking at keep running.

1. **Make a new window:**

   Press `Ctrl-b`, then `c`. (`c` for **create**.)

   You get a fresh prompt. The window you left is still there, still running.

2. **Give it a name** so you can tell them apart:

   Press `Ctrl-b`, then `,` — type a name, press Enter.

3. **Switch between windows:**

   - `Ctrl-b` then `w` — a list of all your windows; arrow keys and Enter.
   - `Ctrl-b` then `0`, `1`, `2` … — jump straight to a window by number.
   - `Ctrl-b` then `n` — the next window.

4. **Close a window** when its work is done: type `exit` in it.

A practical arrangement: window 0 for the curriculum document, window 1 for the website, window 2 for a shell to run commands in. Claude Code can run in several windows at once, working on different things.

Recipe 8 — When something looks wrong

Work through these in order.

What you see	What to do
`ssh` hangs, or "connection refused"	The VPN is not connected. Open WireGuard and activate the tunnel.
`Permission denied (publickey)`	The box does not recognise your key. Check you are using the right username, then ask whoever added your key.
You typed `exit` and lost everything	`exit` closes a window. If it was your last one, tmux ends the session. Next time detach with `Ctrl-b d` instead.
`tmux attach` says "no sessions"	Nothing is running to attach to. Start one: `tmux new -As <project>`.
Your terminal scrollbar does nothing	tmux keeps its own history. Press `Ctrl-b` then `[` to scroll, `q` to leave. The mouse wheel also works.
`git push` is rejected	Someone else pushed first. Run `git pull`, resolve anything it reports, then `git push` again.
Something is slow, or a command was killed	You may have hit your memory limit — see Limits in Part 3.
Your laptop is lost or stolen	Recipe 9, immediately. Two things: tell whoever runs the box, and delete the key from your own GitHub yourself.

If none of these fit, ask Claude Code before asking a person. Paste the exact error message; it can usually read the situation faster than a description of it.

Recipe 9 — If your laptop is lost or stolen

Do these the same hour. Not tomorrow, not after you have looked for it properly. Your key has no passphrase — by deliberate choice, because typing one several times a day costs more than it protects — so how fast the key is revoked **is** the protection.

1. **Tell the person who runs the box.** A message is enough — say which laptop it was. They remove your key from every box and cancel your VPN access. You do not need to do anything for either, and it takes them minutes.
2. **Delete the key from your own GitHub — yourself.** Go to github.com/settings/keys from any other device you can log in on; a phone is fine. Find the key named for the lost laptop and delete it.

   Nobody else can do this step for you. SSH keys belong to your personal GitHub account, and not even an organisation owner can remove them. Until you do it, whoever holds your laptop can still push to every repository you can reach — including the website, which deploys itself.

Then, when you have a working machine again, generate a fresh key pair and go through Recipe 1 from Step 4: new key, register the public half with GitHub, send the same public half for box access.

> **Your files are safe, and nothing is deleted.** Revoking access is not offboarding. Your account on the box, your projects and everything in your home directory stay exactly as they were — you simply cannot log in until a replacement key is registered. Anything you had already pushed is on GitHub regardless.

> Reporting a lost laptop is not an admission that you were careless. Delaying it is the only version of this that causes harm.

Part 3 — Reference

tmux, every key you need

All of these start with `Ctrl-b` — press it, release, then press the second key.

Keys	What it does
Windows — one per piece of work	
`Ctrl-b c`	**Create** a new window. This is how you start a second task.
`Ctrl-b ,`	Rename the current window
`Ctrl-b w`	List all windows and choose one
`Ctrl-b 0–9`	Jump straight to that window
`Ctrl-b n` / `p`	Next / previous window
`Ctrl-b l`	Back to the window you were in last — like alt-tab
`Ctrl-b &`	Close the current window (asks first). Typing `exit` does the same.
Coming and going	
`Ctrl-b d`	**Detach** — leave everything running and return to your laptop
`tmux attach`	Come back to it (typed as a command, not a `Ctrl-b` key)
`tmux new -As <name>`	Attach to a session, creating it if needed
`tmux ls`	List your sessions
Sessions — one per project	
`Ctrl-b s`	List sessions and switch between them
`Ctrl-b $`	Rename the current session
Reading output	
`Ctrl-b [`	Scroll back through output. Arrows and PageUp/PageDown move; `/` searches; `q` returns to the live view.
`Ctrl-b ?`	Every key binding, in case you forget this table

> **Why the terminal scrollbar does nothing.** tmux redraws the whole screen in place, so your terminal application never accumulates any history — each window's scrollback lives inside tmux and is reachable only with `Ctrl-b [`. The mouse wheel is configured to enter that mode automatically. One exception: full-screen programs, Claude Code included, may handle the wheel themselves — `Ctrl-b [` always works.

git, every command you need

Command	What it does
`git pull`	Bring everyone else's latest work down. Do this first, always.
`git status`	What have I changed? Run it constantly; it is free.

Command	What it does
`git diff`	Show the changes line by line
`git add .`	Mark everything you changed as ready to record
`git commit -m "..."`	Record it **on the box**, with a message
`git push`	Send your recorded work **to GitHub**, where others see it
`git log --oneline -10`	The last ten changes, by anyone
`git clone <url>`	Get a project onto the box (once per project)

The daily loop is four of them, in this order:

```
box-01$ git pull                              # start from the latest
box-01$ git status                            # check what you changed
box-01$ git add . && git commit -m "..."      # record it
box-01$ git push                              # share it
```

What is already on the box

Set up before your first login, the same for everyone:

Item	What it means for you
Your own home directory	`0750` — nobody else on the box can read your files, and you cannot read theirs.
`~/src`, `~/wt`, `~/bin`	Empty and ready. Projects go in `~/src`.
tmux, with a starting configuration	Edit `~/.tmux.conf` however you like; it is yours.
Claude Code	Installed. You log in with **your own** account on first use.
Version managers — SDKMAN, pnpm, uv	For installing Java, Node and Python **per project**. The box installs no runtime itself.
PHP, Composer, `gh`	Box-wide, from the distribution.
Your own Docker	`docker` works and is yours alone — your containers and images are invisible to other users.
`DEV_PORT_BASE`	Your own block of 100 port numbers, so two people running a web server never collide.
Lingering	The setting that lets your tmux keep running after you disconnect. Already on.

What you bring

The box deliberately holds no runtime version and none of your credentials:

- **Your Claude Code login** — yours, not a shared one.
- **Runtimes, per project, through the managers** — `sdk env install` from a project's `.sdkmanrc`, `pnpm env use --global <version>`, `uv python install`. Which version is the project's decision, recorded in the project.
- **Any cloud or service credentials** your work needs — keep them in your home directory, never in a repository.

Yours and shared

Shared by everyone	Strictly yours
The machine, the operating system, PHP, `gh`	Your files, projects and documents
The network path in and out	Your Docker containers, images and volumes
The image cache	Your runtimes and their versions
The automatic update timer	Your tmux and every Claude Code session

Limits

The box shares a fixed amount of memory and disk between everyone. Your share is enforced, so one person cannot take the whole machine. Two things happen when you approach it:

- **Everything goes slow and nothing errors.** You are near the memory limit and the machine is throttling you. Close what you are not using.
- **A command dies with no explanation.** You passed the limit and it was stopped. Run fewer things at once.
- **`No space left on device.`** Your disk quota. `quota -s` shows your usage. Delete build output and caches first — they are rebuildable.

> **`DEV_PORT_BASE` is empty in a non-login shell.** It and `DOCKER_HOST` are set by `/etc/profile.d/dev-platform.sh`, which only login shells read. An interactive session has them; `ssh box 'docker compose up'` does not, and the compose file then fails on `${DEV_PORT_BASE:?}` — correctly, but the message points at compose rather than at the shell. Wrap non-interactive commands:
>
> ```
> ssh box 'bash -lc "docker compose up"'
> ```
>
> Rootless Docker still works either way: the setup writes a `rootless` docker context, which does not depend on the environment.

Leaving

When an account is removed, its home directory goes with it — projects, documents, history, everything. Nothing is backed up. `git push` anything you want to keep **before** your last day.

Folder layout

```
~/src/                      # projects, one folder each
  handbook/
  website/
~/wt/                       # worktrees — see Part 4
~/.local/share/docker/      # your own container images
```

Part 4 — Building applications

Everything below is for people who write and run software on the box. If your work is documents and website content, Part 3 is where the guide ends for you.

Two words used precisely

- **Seat** — one developer actively working: their agent session, their running application, and the build that may fire at any moment.

- **Stack** — one project's running services for one task: the application containers plus its dependencies (database, brokers, monitoring).

A seat normally drives one stack. A developer juggling two tasks drives two. How many fit on a given box is a number in the Hardware Specification, not a different way of working — the keystrokes are identical on every box.

Worktrees — one folder per task

`~/src/<project>` is the canonical checkout and is **read-only by rule**. Actual work happens in a worktree: a second working folder for the same repository, on its own branch, so two tasks never fight over one directory.

```
~/src/handbook/          # canonical, tracks main, never edited
~/wt/handbook-rps-2026/  # one task
~/wt/website-hero-redesign/  # another task, different project
```

The `<project>-<task>` prefix also keeps container names and compose projects collision-free. Per-worktree build output — `target/`, `node_modules/`, `vendor/`, `.venv/` — is disposable churn, rebuilt from the shared caches.

Put this in every project's `CLAUDE.md`, so each session starts under the rule:

```
## Working on this repository on a development box
- All implementation work happens in a worktree under `~/wt/<project>-<slug>`,
  never in this checkout (`~/src/<project>`), which tracks `main` read-only.
- Publish ports from `${DEV_PORT_BASE:?}`; never hardcode a host port.
- Runtime versions: `.sdkmanrc` / `.nvmrc` / `.python-version` in this repo
  are authoritative — install through the managers, not system packages.
```

Dependency caches

One principle per package manager: **one durable shared cache per user under `$HOME`; per-worktree install directories are disposable.** Worktree-heavy work multiplies install directories — without a shared store, N worktrees mean N downloads and N disk copies.

- **Java / Maven** — `~/.m2/repository` is yours and every worktree shares it.
- **Node** — **pnpm only, never npm.** pnpm keeps one content-addressable store and hardlinks into each worktree; after the first install the rest are near instant.
- **Python** — **uv only, never system pip.**
- **PHP** — Composer's cache works like `~/.m2`. `vendor/` is per-worktree churn.

Publishing ports

Rootless Docker isolates container networks, but published ports land in the host's single port space — two developers running a web app both want `:8080`. Use your block, and make the compose file fail if it is unset:

```
services:
  app:
    ports:
      - "${DEV_PORT_BASE:?DEV_PORT_BASE unset — see the Developer Guide}:8080"
```

Testcontainers binds random ephemeral ports and needs no block; this convention is only for services a human reaches by hand.

Session model

Session = project, window = task. `Ctrl-b s` switches projects the way `Ctrl-b w` switches tasks within one — two levels, and that is the whole hierarchy. A separate `ops` session holds what belongs to no project.

```
laptop$ ssh box-01
box-01$ tmux new -As handbook
box-01$ tmux neww -n rps-2026
box-01$ cd ~/src/handbook
box-01$ claude
```

When a task finishes, its Claude Code session ends and its window closes; the project session keeps hosting the rest. A new task is a new window, a new `claude`, a new worktree; a new project is a new session.

Concurrency rules

- **Your budget is enforced, not agreed.** Exceed it and you get a stopped process, not a slow box for everybody else.
- **A task owns its slice across modules of its project.** If two of your active tasks in the same project both need the shared module, serialize them.
- **Two developers do not take the same project's shared module concurrently.** This is the only coordination that crosses user boundaries, and it is social, not technical.
- **When every seat is full, the answer is another box**, not a tighter squeeze.

Standing test loops

A project can have a standing loop: every 15 minutes, under its own service account, the box fetches the project's `main`, and if it moved, runs the project's full end-to-end suite. Green is silent; red posts a notification with the project and the commit.

The box only **runs** the suite; the project **defines** it. The repository must provide `bin/e2e.sh` at its root, which brings the stack up, runs everything, tears it down, and exits non-zero on failure. Its absence is a hard failure by design: a project that silently never verifies is worse than one that fails loudly on the first run.

What does not run on these boxes

- Performance testing — that belongs on rented cloud machines. This box is for functional correctness.
- Anything public-facing. There are no inbound ports; access is through the VPN or the campus LAN only.
- Anything that is not development work.